1. MMIO vs DMA speedup How much faster is DMA?Why?

3. MMIO FIR (one sample per round-trip)

Each sample requires: write input → start IP → poll done → read output.

Note: Check the register offsets against the HLS driver header (`xfir_mmio_hw.h`) after synthesis.

In [13]:
```
# Register offsets (from HLS synthesis report -- verify after build)
CTRL_REG       = 0x00
X_IN_OFFSET    = 0x18
RETURN_OFFSET  = 0x10

def float_to_uint(f):
    return struct.unpack('<I', struct.pack('<f', f))[0]

def uint_to_float(u):
    return struct.unpack('<f', struct.pack('<I', u & 0xFFFFFFFF))[0]

def mmio_fir_one_sample(ip, x_val):
    ip.write(X_IN_OFFSET, float_to_uint(x_val))
    ip.write(CTRL_REG, 0x01)                    # ap_start
    while (ip.read(CTRL_REG) & 0x02) == 0:      # wait ap_done
        pass
    return uint_to_float(ip.read(RETURN_OFFSET))
```

In [14]:
```
y_mmio = np.zeros(N_SAMPLES, dtype=np.float32)

t0 = time.perf_counter()
for i in range(N_SAMPLES):
    y_mmio[i] = mmio_fir_one_sample(fir_mmio_ip, float(x[i]))
t_mmio = time.perf_counter() - t0

print(f"MMIO: {N_SAMPLES} samples in {t_mmio*1e3:.2f} ms "
      f"({t_mmio/N_SAMPLES*1e6:.1f} us/sample)")
```

```
MMIO: 256 samples in 34.01 ms (132.8 us/sample)
```

4. DMA Streaming FIR

Data path: PS → DMA TX → AXI-Stream → FIR → AXI-Stream → DMA RX → PS

The CPU only sets up the transfer and waits -- the FPGA processes all samples autonomously.

In [15]:
```
in_buf  = allocate(shape=(N_SAMPLES,), dtype=np.float32)
out_buf = allocate(shape=(N_SAMPLES,), dtype=np.float32)

np.copyto(in_buf, x)
out_buf[:] = 0

# Configure streaming FIR: set sample count and start
# Register 0x10 = 'n' parameter (check synthesis report)
fir_stream.write(0x10, N_SAMPLES)
fir_stream.write(0x00, 0x01)   # ap_start

t0 = time.perf_counter()
dma.sendchannel.transfer(in_buf)
dma.recvchannel.transfer(out_buf)
dma.sendchannel.wait()
dma.recvchannel.wait()
t_dma = time.perf_counter() - t0

y_dma = np.array(out_buf, dtype=np.float32)
print(f"DMA: {N_SAMPLES} samples in {t_dma*1e3:.2f} ms "
      f"({t_dma/N_SAMPLES*1e6:.1f} us/sample)")
```

```
DMA: 256 samples in 3.75 ms (14.7 us/sample)
```

DMA Streaming takes : 3.75 ms

MMIO takes :34.01 ms

Reason:

- **MMIO**: CPU transfers data one word at a time, so every transfer costs instructions + bus access → slow and CPU-heavy
- **DMA**: CPU only sets up once, then hardware moves data in large bursts directly → fast and efficient

Why DMA is much faster:

- No per-word CPU overhead
- Uses burst transfers (better bandwidth)
- Runs in parallel with the CPU

2. Unroll factor What happens when you remove#pragma HLS UNROLL from the MAC loop?How does II change?

When I did the HLS synthesis with #pragma HLS UNROLL, I got the II=1 in both cases of floating point implementation and fixed point implementation.

Report of Floationg point representation and UNROLLED:

```
================================================================
== Synthesis Summary Report of 'fir_stream'
================================================================
+ General Information:
    * Date:           Thu Apr  9 22:39:48 2026
    * Version:        2021.1 (Build 3247384 on Thu Jun 10 19:36:07 MDT 2021)
    * Project:        new_hls_stream
    * Solution:       solution1 (Vivado IP Flow Target)
    * Product family: zynq
    * Target device:  xc7z020-clg400-1


+ Performance & Resource Estimates:

    PS: '+' for module; 'o' for loop; '*' for dataflow
```

```
+-----------------+------+------+--------+--------+---------+--------+------+---------+------+----------+-----------+
-----------+-----+
   |      Modules      | Issue|      | Latency | Latency| Iteration|         | Trip |         |     |          |
|           |     |
   |      & Loops      | Type | Slack| (cycles)|  (ns)  |  Latency | Interval| Count| Pipelined|
BRAM |    DSP    |      FF     |     LUT     | URAM|

+-----------------+------+------+--------+--------+---------+--------+------+---------+------+----------+-----------+
-----------+-----+
   |+ fir_stream       |    -|  0.04|       -|      -|        -|       -|    -|       no|    -|  105 (47%)|  11732
(11%)|  16144 (30%)|    -|
   | o VITIS_LOOP_25_1  |    -|  7.30|       -|      -|      111|       1|    -|      yes|    -|         -|
-|          -|    -|

+-----------------+------+------+--------+--------+---------+--------+------+---------+------+----------+-----------+
-----------+-----+


================================================================
== HW Interfaces
================================================================
* S_AXILITE
+-----------+-----------+-------------+--------+---------+
| Interface  | Data Width | Address Width | Offset | Register |
+-----------+-----------+-------------+--------+---------+
| s_axi_CTRL | 32        | 5           | 16     | 0       |
+-----------+-----------+-------------+--------+---------+

* AXIS
+-----------+--------------+-------+-------+-----+-------+-------+--------+-------+-------+--------+
| Interface  | Register Mode | TDATA | TDEST | TID | TKEEP | TLAST | TREADY | TSTRB |
TUSER | TVALID |
+-----------+--------------+-------+-------+-----+-------+-------+--------+-------+-------+--------+
| in_stream  | both          | 32    | 1     | 1   | 4     | 1     | 1      | 4     | 1     | 1      |
| out_stream | both          | 32    | 1     | 1   | 4     | 1     | 1      | 4     | 1     | 1      |
+-----------+--------------+-------+-------+-----+-------+-------+--------+-------+-------+--------+

* TOP LEVEL CONTROL
+-----------+-----------+-----------+
| Interface | Type       | Ports     |
+-----------+-----------+-----------+
| ap_clk    | clock      | ap_clk    |
| ap_rst_n  | reset      | ap_rst_n  |
```

```
| interrupt | interrupt  | interrupt |
| ap_ctrl   | ap_ctrl_hs |           |
+-----------+------------+-----------+


================================================================
== SW I/O Information
================================================================
* Top Function Arguments
+------------+-----------+-----------------------------------------+
| Argument   | Direction | Datatype                                |
+------------+-----------+-----------------------------------------+
| in_stream  | in        | stream<hls::axis<ap_int<32>, 1, 1, 1>, 0>& |
| out_stream | out       | stream<hls::axis<ap_int<32>, 1, 1, 1>, 0>& |
| n_samples  | in        | int                                     |
+------------+-----------+-----------------------------------------+

* SW-to-HW Mapping
+------------+--------------------+-----------+---------------------+
| Argument   | HW Name            | HW Type   | HW Info             |
+------------+--------------------+-----------+---------------------+
| in_stream  | in_stream          | interface |                     |
| out_stream | out_stream         | interface |                     |
| n_samples  | s_axi_CTRL n_samples | register  | offset=0x10, range=32 |
+------------+--------------------+-----------+---------------------+
```

Report of Fixed point representation and UNROLLED:

```
================================================================
== Synthesis Summary Report of 'fir_stream'
================================================================
+ General Information:
    * Date:           Fri Apr 17 21:47:03 2026
    * Version:        2021.1 (Build 3247384 on Thu Jun 10 19:36:07 MDT 2021)
    * Project:        new_hls_stream
    * Solution:       solution1 (Vivado IP Flow Target)
    * Product family: zynq
    * Target device:  xc7z020-clg400-1


+ Performance & Resource Estimates:
```

PS: '+' for module; 'o' for loop; '*' for dataflow

```
+-----------------------------------------+------+------+---------+--------+----------+---------+------+----------+------+-
--------+-----------+----------+-----+
   |                Modules                | Issue|      | Latency | Latency| Iteration|         | Trip |
|     |        |           |          |     |
   |                & Loops                | Type | Slack| (cycles)|  (ns)  |  Latency | Interval|
Count| Pipelined| BRAM |   DSP   |     FF     |    LUT    | URAM|

+-----------------------------------------+------+------+---------+--------+----------+---------+------+----------+------+-
--------+-----------+----------+-----+
   |+ fir_stream                           |    -|  0.23|       -|      -|        -|       -|    -|       no|    -|  11
(5%)|   1193 (1%)|  1025 (1%)|    -|
   | + grp_fir_stream_Pipeline_sample_loop_fu_120  |    -|  0.48|       -|      -|        -|       -|
-|       no|    -|  11 (5%)|  1019 (~0%)|   829 (1%)|    -|
   |  o sample_loop                        |    -|  7.30|       -|      -|       15|       1|    -|      yes|    -|
-|         -|        -|    -|

+-----------------------------------------+------+------+---------+--------+----------+---------+------+----------+------+-
--------+-----------+----------+-----+


================================================================
== HW Interfaces
================================================================
* S_AXILITE
+-------------+-----------+-------------+--------+----------+
| Interface     | Data Width | Address Width | Offset | Register |
+-------------+-----------+-------------+--------+----------+
| s_axi_control | 32         | 5             | 16     | 0        |
+-------------+-----------+-------------+--------+----------+

* AXIS
+-----------+--------------+-------+-------+-----+-------+-------+--------+-------+-------+--------+
| Interface  | Register Mode | TDATA | TDEST | TID | TKEEP | TLAST | TREADY | TSTRB |
TUSER | TVALID |
+-----------+--------------+-------+-------+-----+-------+-------+--------+-------+-------+--------+
| in_stream  | both          | 32    | 1     | 1   | 4     | 1     | 1      | 4     | 1     | 1      |
| out_stream | both          | 32    | 1     | 1   | 4     | 1     | 1      | 4     | 1     | 1      |
+-----------+--------------+-------+-------+-----+-------+-------+--------+-------+-------+--------+

* TOP LEVEL CONTROL
+-----------+-----------+-----------+
| Interface | Type       | Ports     |
```

```
+----------+-----------+----------+
| ap_clk    | clock      | ap_clk    |
| ap_rst_n  | reset      | ap_rst_n  |
| interrupt | interrupt  | interrupt |
| ap_ctrl   | ap_ctrl_hs |           |
+----------+-----------+----------+


================================================================
== SW I/O Information
================================================================
* Top Function Arguments
+-----------+----------+-----------------------------------------+
| Argument   | Direction | Datatype                                |
+-----------+----------+-----------------------------------------+
| in_stream  | in        | stream<hls::axis<ap_int<32>, 1, 1, 1>, 0>& |
| out_stream | out       | stream<hls::axis<ap_int<32>, 1, 1, 1>, 0>& |
| n          | in        | int                                     |
+-----------+----------+-----------------------------------------+

* SW-to-HW Mapping
+-----------+---------------+----------+--------------------+
| Argument   | HW Name        | HW Type   | HW Info             |
+-----------+---------------+----------+--------------------+
| in_stream  | in_stream      | interface |                    |
| out_stream | out_stream     | interface |                    |
| n          | s_axi_control n | register  | offset=0x10, range=32 |
+-----------+---------------+----------+--------------------+
```

But when we remove the pragma UNROLLED, the II got changed to 23, as it got started executing step-by-step, not parallely.

```
================================================================
== Synthesis Summary Report of 'fir_stream'
================================================================
```

+ General Information:
    * Date: Thu Apr 9 22:41:56 2026
    * Version: 2021.1 (Build 3247384 on Thu Jun 10 19:36:07 MDT 2021)
    * Project: new_hls_stream
    * Solution: solution2 (Vivado IP Flow Target)
    * Product family: zynq
    * Target device: xc7z020-clg400-1

+ Performance & Resource Estimates:

    PS: '+' for module; 'o' for loop; '*' for dataflow

+---+------+------+---------+---------+----------+---------+------+----------+-----+-----------+------------+------------+-----+
    | Modules | Issue| | Latency | Latency | Iteration| | Trip |
| | | | | |
    | & Loops | Type | Slack| (cycles)| (ns) | Latency | Interval|
Count| Pipelined| BRAM | DSP | FF | LUT | URAM|

+---+------+------+---------+---------+----------+---------+------+----------+-----+-----------+------------+------------+-----+
    |+ fir_stream | -| 0.04| -| -| -| -| -| no| -|
105 (47%)| 11734 (11%)| 22878 (43%)| -|
    | + grp_fir_stream_Pipeline_1_fu_152 | -| 0.32| 23| 230.000| -|
23| -| no| -| -| 658 (~0%)| 6961 (13%)| -|
    | o Loop 1 | -| 7.30| 21| 210.000| 2| 1| 21|
yes| -| -| -| -| -|
    | + grp_fir_stream_Pipeline_VITIS_LOOP_25_1_fu_176 | -| 0.04| -| -| -|
-| -| no| -| 105 (47%)| 10963 (10%)| 15771 (29%)| -|
    | o VITIS_LOOP_25_1 | -| 7.30| -| -| 110| 1| -|
yes| -| -| -| -| -|

+---+------+------+---------+---------+----------+---------+------+----------+-----+-----------+------------+------------+-----+

==
== HW Interfaces
==
* S_AXILITE
+-----------+------------+--------------+--------+----------+
| Interface | Data Width | Address Width | Offset | Register |
+-----------+------------+--------------+--------+----------+

| s_axi_CTRL | 32 | 5 | 16 | 0 |
+------------+-----------+--------------+--------+---------+

* AXIS
+------------+--------------+-------+-------+-----+-------+-------+--------+-------+-------+--------+
| Interface | Register Mode | TDATA | TDEST | TID | TKEEP | TLAST | TREADY | TSTRB | TUSER | TVALID |
+------------+--------------+-------+-------+-----+-------+-------+--------+-------+-------+--------+
| in_stream | both | 32 | 1 | 1 | 4 | 1 | 1 | 4 | 1 | 1 | 1 |
| out_stream | both | 32 | 1 | 1 | 4 | 1 | 1 | 4 | 1 | 1 | 1 |
+------------+--------------+-------+-------+-----+-------+-------+--------+-------+-------+--------+

* TOP LEVEL CONTROL
+-----------+-----------+-----------+
| Interface | Type | Ports |
+-----------+-----------+-----------+
ap_clk	clock	ap_clk
ap_rst_n	reset	ap_rst_n
interrupt	interrupt	interrupt
ap_ctrl	ap_ctrl_hs	
+-----------+-----------+-----------+

==
== SW I/O Information
==
* Top Function Arguments
+------------+-----------+---+
| Argument | Direction | Datatype |
+------------+-----------+---+
in_stream	in	stream<hls::axis<ap_int<32>, 1, 1, 1>, 0>&
out_stream	out	stream<hls::axis<ap_int<32>, 1, 1, 1>, 0>&
n_samples	in	int
+------------+-----------+---+

* SW-to-HW Mapping
+------------+--------------------+-----------+--------------------+
| Argument | HW Name | HW Type | HW Info |
+------------+--------------------+-----------+--------------------+
in_stream	in_stream	interface	
out_stream	out_stream	interface	
n_samples	s_axi_CTRL n_samples	register	offset=0x10, range=32
+------------+--------------------+-----------+--------------------+

3. With 21 taps fully unrolled, how many DSPs are used?

With 21 taps fully unrolled 105 DSPs were used in floating point representation.

4. Switch to ap_fixed16,4. How do resources and timing change?Is the filter output still correct?

By transitioning the design from **floating-point** to a **fixed-point** architecture ,we achieved a significant reduction in hardware resource utilization, specifically regarding digital signal processing (DSP) slices.

In case of <ap_fixed 16,4> , **it was taking 11 DSPs ,(due to symmetricity of 21 Tap filter).**

In case of floating point it was **105 DSPs** , as 1 Floating point multiplier consumes **5 DSPs so 21 tap filter will approximately uses 21*5 =105 DSPs.**

5. AXI Timer Compare Python timing with cycle-accurate HW timing.What is the overhead of the Python/DMA setup?

- Timing measured in Python reflects the **overall latency of the system**, not just the hardware execution. It includes time spent in the Python runtime, operating system scheduling, driver interactions, and the setup of the DMA transfer.
- It is **not cycle-accurate** and typically falls in the **µs–ms range**.
- **Hardware (cycle-accurate) timing** measures exact clock cycles inside the FPGA → **precise and deterministic**.
- It reflects only the **true performance of the hardware block**.